P.S.V. COLLEGE OF ENGINEERING AND TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

MITTAPALLI, BALINAYANAPALLI (PO), KRISHNAGIRI – 635 108

DEPARTMENT OF CSE(CYBER SECURITY)

CS3591

COMPUTER NETWORKS LABORATORY

III YEAR/V SEM

LAB MANUAL

R2021

Ex.No.1 Learn to use commands like tcpdump, netstat, ifconfig, nslookup and trace route. Capture ping and trace route PDUs using a network protocol analyzer and examine

Aim:

To use commands like tcpdump, netstat, ifconfig, nslookup and trace route. Capture ping and trace route PDUs using a network protocol analyzer and examine.

1. Tcpdump

Tcpdump is a command line utility that allows you to capture and analyze network traffic going through your system.

Procedure

Check if tcpdump is installed on your system

$ which tcpdump

/usr/sbin/tcpdump

If tcpdump is not installed,

$ sudo apt install tcpdump

To get Supervisor Privilege

$ su

(and password 123456)
$ sudo –i to change #
($ is changed to # and the commands can be executed in supervisor)

Capturing packets with tcpdump

Use the command tcpdump -D to see which interfaces are available for capture.

[root@localhost cse]# tcpdump -D
1. nflog (Linux netfilter log (NFLOG) interface)
2. nfqueue (Linux netfilter queue (NFQUEUE) interface)
3. usbmon1 (USB bus number 1)
4. enp2s0
5. usbmon2 (USB bus number 2)
6. any (Pseudo-device that captures on all interfaces)
7. lo [Loopback]

Capture all packets in any interface by running this command:

[root@localhost cse]# tcpdump -i any

06:03:58.258143 ARP, Request who-has 172.16.51.87 tell 172.16.22.25, length 46
06:03:58.258225 ARP, Request who-has 172.16.51.88 tell 172.16.22.25, length 46
06:03:58.260828 ARP, Request who-has 172.16.51.122 tell 172.16.22.25, length 46
06:03:58.260903 ARP, Request who-has 172.16.51.123 tell 172.16.22.25, length 46
^C
5244 packets captured
59636 packets received by filter
54378 packets dropped by kernel
(Press ctrl+C to stop execution)

Filter packets based on the source or destination IP Address

[root@localhost cse]#tcpdump -i any -c5 -nn src 172.16.20.138

6:10:30.712414 ARP, Request who-has 172.16.16.16 tell 172.16.20.138, length 28

06:10:31.483765 IP 172.16.20.138.47997 > 51.158.186.98.123: NTPv4, Client, length 48

5 packets captured

5 packets received by filter

0 packets dropped by kernel

[root@localhost cse]#tcpdump -i any -c5 -nn dst 172.16.20.139

6:10:30.712414 ARP, Request who-has 172.16.16.16 tell 172.16.20.138, length 28

06:10:31.483765 IP 172.16.20.138.47997 > 51.158.186.98.123: NTPv4, Client, length 48

5 packets captured

5 packets received by filter

0 packets dropped by kernel

Filtering packets

To filter packets based on protocol, specifying the protocol in the command line. For example, capture ICMP packets only by using this command:

[root@localhost cse]# tcpdump -i any -c5 icmp

(tcpdump captures and displays only the ICMP-related packets.)

tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on any, link-type LINUX_SLL (Linux cooked), capture size 262144 bytes
06:15:07.800786 IP localhost.localdomain > ec2-54-204-39-132.compute-1.amazonaws.com: ICMP echo request, id 8180, seq 13, length 64
06:15:08.063488 IP ec2-54-204-39-132.compute-1.amazonaws.com > localhost.localdomain: ICMP echo reply, id 8180, seq 13, length 64

5 packets captured
5 packets received by filter
0 packets dropped by kernel

In a different terminal, try to ping another machine:
$ ping opensource.com

2. netstat

netstat (network statistics) is a command line tool for monitoring network connections both incoming and outgoing as well as viewing routing tables, interface statistics etc.

[root@localhost cse]# netstat
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address Foreign Address State
tcp 0 0 localhost.localdo:53318 ec2-52-206-98-166:https ESTABLISHED
tcp 0 0 localhost.localdo:36418 sg2plpkivs-v03.any:http TIME_WAIT

-at → list all TCP ports
-au → list all UDP ports
-l → listening ports
-lt → listening TCP
-lu → listening UDP
-s → statistics of all ports
-su →statistics of UDP
-st → statistics of TCP

3. ifconfig

It displays the details of a network interface card like IP address, MAC Address, and the status of a network interface card

[cse@localhost ~]$ ifconfig

enp2s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 172.16.20.138 netmask 255.255.0.0 broadcast 172.16.255.255
inet6 fe80::d884:13bc:fd22:2d43 prefixlen 64 scopeid 0x20<link>
ether a0:8c:fd:e7:10:86 txqueuelen 1000 (Ethernet)
RX packets 4474083 bytes 280780119 (267.7 MiB)
RX errors 0 dropped 353 overruns 0 frame 0
TX packets 14455 bytes 1798944 (1.7 MiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>

3

loop txqueuelen 1000 (Local Loopback)
RX packets 4154 bytes 352264 (344.0 KiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 4154 bytes 352264 (344.0 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

4. nslookup

nslookup (stands for "Name Server Lookup") is a useful command for getting information from DNS server. It is a network administration tool for querying the Domain Name System (DNS) to obtain domain name or IP address mapping or any other specific DNS record.

[cse@localhost ~]$ nslookup annauniv.edu
Server: 8.8.8.8
Address: 8.8 8.8#53

Non-authoritative answer:
Name: annauniv.edu
Address: 103.70.60.38

[cse@localhost ~]$ nslookup 172.217.26.206
Server: 8.8.8.8
Address: 8.8 8.8#53

Non-authoritative answer:
206.26.217.172.in-addr.arpa name = maa03s23-in-f14.1e100.net.
206.26.217.172.in-addr.arpa name = maa03s23-in-f14.1e100.net.
206.26.217.172.in-addr.arpa name = maa03s23-in-f206.1e100.net.
206.26.217.172.in-addr.arpa name = maa03s23-in-f206.1e100.net.

Authoritative tracerouteanswers can be found from:

Lookup for any record
[cse@localhost ~]$ nslookup -type=any annauniv.edu
Server: 8.8.8.8
Address: 8.8 8.8#53

Non-authoritative answer:
Name: annauniv.edu
Address: 103.70.60.38
annauniv.edu text = "v=spf1 ip4:103.70.60.40 -all"
annauniv.edu mail exchanger = 0 sonic.annauniv.edu.
annauniv.edu
origin = ns.annauniv.edu
mail addr = root.annauniv.edu
serial = 20170907
refresh = 300

4

retry = 900
expire = 604800
minimum = 86400
annauniv.edu nameserver = ns.annauniv.edu.

Authoritative answers can be found from:

Lookup for an ns record

[cse@localhost ~]$ nslookup -type=ns annauniv.edu
Server: 8.8.8.8
Address: 8.8 8.8#53

Non-authoritative answer:
annauniv.edu nameserver = ns.annauniv.edu.

Authoritative answers can be found from

5. traceroute

The **traceroute** command is used in **Linux** to map the journey that a packet of information undertakes from its source to its destination.

[cse@localhost ~]$ traceroute

Usage:
traceroute [-46dFITnreAUDV] [-f first_ttl] [-g gate, ...] [-i device] [-m max_ttl] [-N squeries] [-p port] [-t tos] [-l flow_label] [-w waittime] [-q nqueries] [-s src_addr] [-z sendwait] [--fwmark=num] host [packetlen]
Options:
-4 Use IPv4
-6 Use IPv6
-d --debug Enable socket level debugging
-F --dont-fragment Do not fragment packets

[cse@localhost ~]$ traceroute annauniv.edu
traceroute to annauniv.edu (103.70.60.38), 30 hops max, 60 byte packets
1 117.193.124.33 (117.193.124.33) 1.389 ms 1.216 ms 1.072 ms
2 172.16.199.74 (172.16.199.74) 1.902 ms 1.834 ms 1.761 ms
3 218.248.235.161 (218.248.235.161) 27.212 ms * *
4 * * *
5 218.248.178.42 (218.248.178.42) 15.521 ms * *
6 * * *
7 madurai-eg-175.232.249.45.powergrid.in (45.249.232.175) 16.007 ms 15.345 ms 15.867 ms

[cse@localhost ~]$ traceroute 172.16.20.139
traceroute to 172.16.20.139 (172.16.20.139), 30 hops max, 60 byte packets

5

1 localhost.localdomain (172.16.20.138) 3004.348 ms !H 3004.215 ms !H 3004.104 ms !H

Capture ping and traceroute PDUs using a network protocol analyzer and examine.

Network protocol analyzer - wireshark

Wireshark is free & Open source network packet analyzer that is used for network analysis, troubleshooting, etc.

Wireshark is quite similar to tcpdump, the major difference between the two is that Wireshark has a graphical interface with built-in filtering options, which make it easy to use.

Installation commands on Wireshark

sudo apt install wireshark

To Open Wireshark

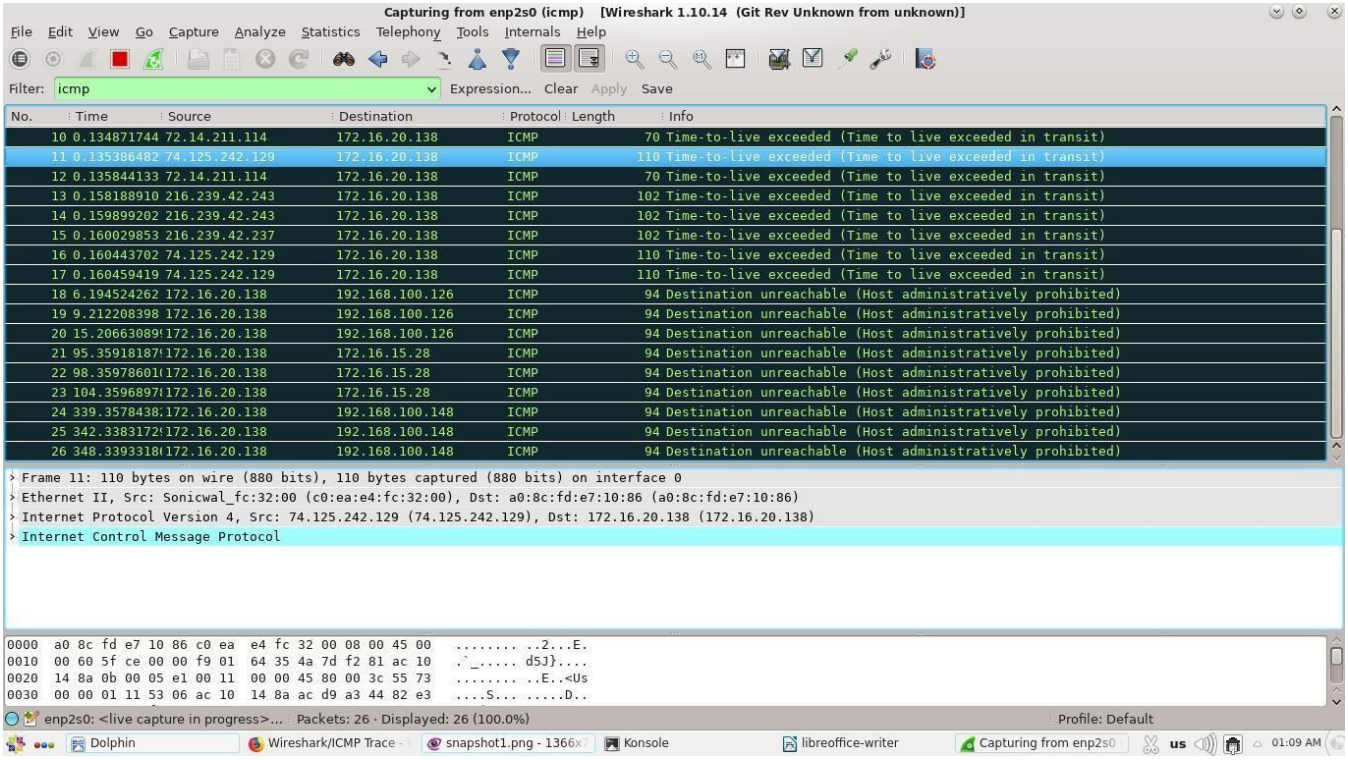
Open directly or use the following commands

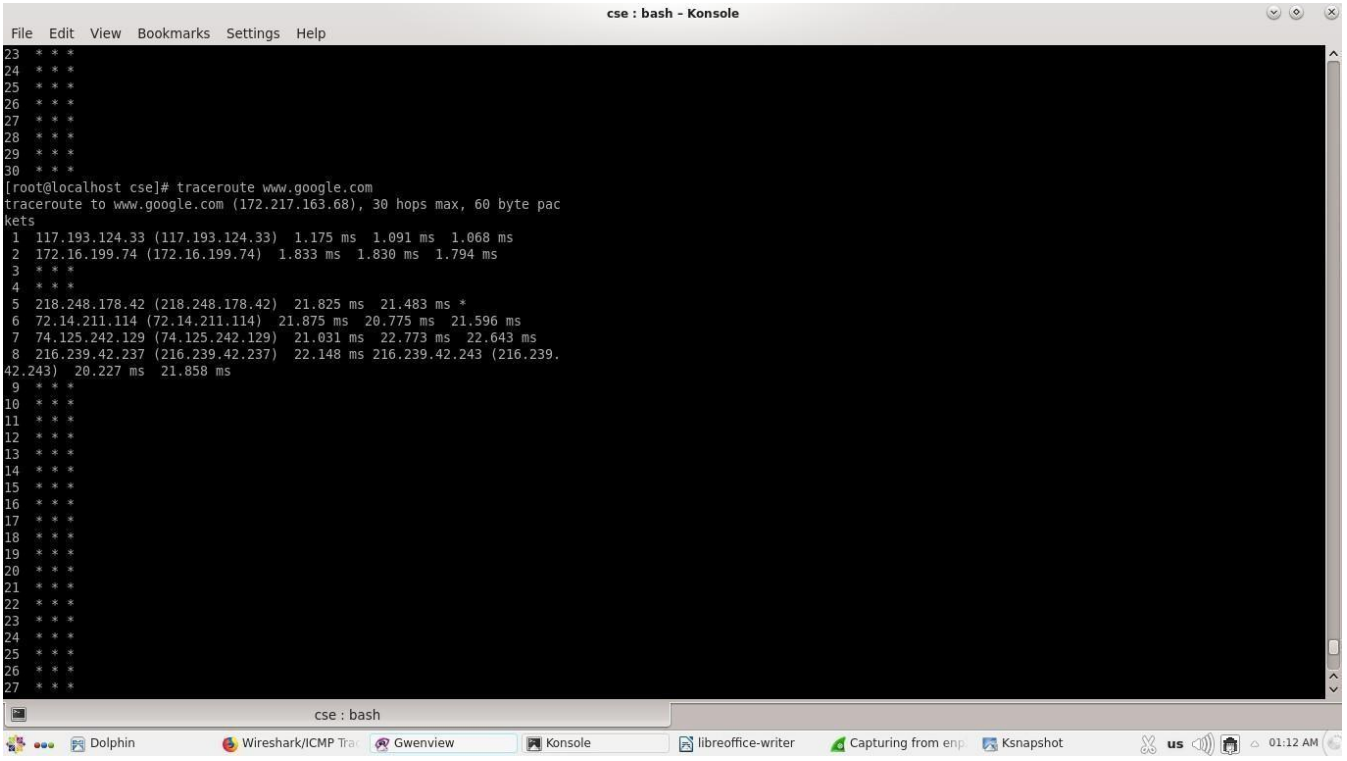
sudo wireshark

In wireshark filter icmp packets

In a konsole execute
ping www.sudo.com
traceroute www.google.com

6





Result:

Thus commands like tcpdump, netstat, ifconfig, nslookup and traceroute was used. Ping and traceroute PDUs using a network protocol analyzer was captured and examined.

7

Ex: 2 WRITE A HTTP WEB CLIENT PROGRAM TO DOWNLOAD A WEB PAGE USING TCP SOCKETS

AIM

To write a program in Python to create an HTTP web client that downloads a web page using TCP sockets.

Algorithm

1. **Start the program**:
   - Import the necessary modules, such as socket.
2. **Define the target server and resource**:
   - Specify the hostname (e.g., example.com) and the resource to download (e.g., /index.html).
3. **Establish a connection**:
   - Create a TCP socket.
   - Connect to the specified server on port 80 (default HTTP port).
4. **Send an HTTP GET request**:
   - Construct and send an HTTP GET request using the socket.
5. **Receive the server's response**:
   - Read data from the socket in chunks until the entire response is received.
6. **Separate headers and body**:
   - Parse the HTTP response to separate headers from the body.
7. **Save the body (webpage content)**:
   - Write the body of the HTTP response (HTML content) to a file in the current working directory.
8. **Stop the program**:
   - Close the socket and terminate the program.

PROGRAM

```
import socket

def download_web_page(host, resource):
    try:
        # Create a TCP socket
        client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # Connect to the server on port 80 (HTTP default port)
        client_socket.connect((host, 80))

        # Formulate the HTTP GET request
```

```
        request = f"GET {resource} HTTP/1.1\r\nHost: {host}\r\nConnection: close\r\n\r\n"
        client_socket.sendall(request.encode())

        # Receive the response data
        response = b""
        while True:
            data = client_socket.recv(4096)
            if not data:
                break
            response += data

        # Close the socket
        client_socket.close()

        # Split the response into headers and body
        headers, _, body = response.partition(b"\r\n\r\n")

        # Save the body (HTML content) to a file
        with open("downloaded_page.html", "wb") as file:
            file.write(body)

        print("Web page downloaded successfully and saved as 'downloaded_page.html'")
    except Exception as e:
        print(f"An error occurred: {e}")

# Main function
if __name__ == "__main__":
    # Specify the host and resource
    host = "example.com"  # Replace with the desired host
    resource = "/"        # Replace with the specific resource path (e.g., "/index.html")

    # Call the function to download the web page
    download_web_page(host, resource)
```

Result

The program successfully downloads a web page using TCP sockets and saves it as an HTML file in the current working directory.

EX NO :3 A. APPLICATION USING TCP SOCKETS - ECHO CLIENT AND ECHO SERVER

Aim

To write a program in Python to implement an Echo Client and Echo Server using TCP sockets where the client sends a message, and the server echoes it back.

Algorithm

Server-Side Algorithm

1. **Start the program**:
   - Import the socket module for TCP communication.
2. **Create a socket**:
   - Use socket.socket() with AF_INET for IPv4 and SOCK_STREAM for TCP.
3. **Bind the socket**:
   - Bind the socket to a specific host (e.g., localhost) and a port (e.g., 12345).
4. **Listen for incoming connections**:
   - Use listen() to wait for client connections.
5. **Accept a connection**:
   - Accept a client connection using accept().
6. **Receive data from the client**:
   - Use recv() to read the client's message.
7. **Echo the message back to the client**:
   - Use sendall() to send the same data back.
8. **Close the connection**:
   - Close the connection after the client disconnects.

Client-Side Algorithm

1. **Start the program**:
   - Import the socket module.
2. **Create a socket**:
   - Use socket.socket() with AF_INET for IPv4 and SOCK_STREAM for TCP.
3. **Connect to the server**:
   - Use connect() to establish a connection with the server's host and port.
4. **Send a message to the server**:
   - Use sendall() to send data to the server.
5. **Receive the echo message**:
   - Use recv() to read the data echoed by the server.
6. **Display the echoed message**:

- Print the server’s response to the console.

7. **Close the socket**:
   - Close the connection after communication.

```
import socket

def start_echo_server(host='localhost', port=12345):
    try:
        # Create a TCP socket
        server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # Bind the socket to the address and port
        server_socket.bind((host, port))
        print(f"Server started at {host}:{port}")

        # Listen for incoming connections
        server_socket.listen(1)
        print("Waiting for a connection...")

        # Accept a connection
        conn, addr = server_socket.accept()
        print(f"Connection established with {addr}")

        while True:
            # Receive data from the client
            data = conn.recv(1024)
            if not data:
                break
            print(f"Received: {data.decode()}")

            # Echo the data back to the client
            conn.sendall(data)

        # Close the connection
        conn.close()
        print("Connection closed.")
    except Exception as e:
        print(f"An error occurred: {e}")
    finally:
        server_socket.close()

# Start the server
if __name__ == "__main__":
    start_echo_server()
```

CLIENT PROGRAM

```
import socket

def start_echo_client(host='localhost', port=12345):
    try:
        # Create a TCP socket
        client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # Connect to the server
        client_socket.connect((host, port))
        print(f"Connected to server at {host}:{port}")

        # Input message to send
        message = input("Enter a message to send: ")
        client_socket.sendall(message.encode())

        # Receive the echo from the server
        data = client_socket.recv(1024)
        print(f"Echoed from server: {data.decode()}")

        # Close the connection
        client_socket.close()
    except Exception as e:
        print(f"An error occurred: {e}")

# Start the client
if __name___== "__main__":
    start_echo_client()
```

3.BAPPLICATION USING TCP SOCKETS - CHAT APPLICATION

Aim

To write a program in Python to implement a simple cB hat application using TCP sockets where a client and server can exchange messages.

Algorithm

Server-Side Algorithm

1. **Start the program**:
    - Import the socket module.
2. **Create a socket**:

- Use socket.socket() with AF_INET for IPv4 and SOCK_STREAM for TCP.

3. **Bind the socket**:
   - Bind the socket to a specific host (e.g., localhost) and port (e.g., 12345).
4. **Listen for incoming connections**:
   - Use listen() to wait for client connections.
5. **Accept a connection**:
   - Accept a client connection using accept().
6. **Exchange messages**:
   - Continuously receive and send messages between the server and client until one of them exits.
7. **Close the connection**:
   - Close the socket after communication ends.

Client-Side Algorithm

1. **Start the program**:
   - Import the socket module.
2. **Create a socket**:
   - Use socket.socket() with AF_INET for IPv4 and SOCK_STREAM for TCP.
3. **Connect to the server**:
   - Use connect() to establish a connection with the server's host and port.
4. **Exchange messages**:
   - Continuously send and receive messages between the client and server until one of them exits.
5. **Close the socket**:
   - Close the connection after communication ends.

SERVER

```
import socket

def chat_server(host='localhost', port=12345):
    try:
        # Create a TCP socket
        server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # Bind the socket to the address and port
        server_socket.bind((host, port))
        print(f"Server started at {host}:{port}")

        # Listen for incoming connections
        server_socket.listen(1)
        print("Waiting for a client to connect...")

        # Accept a connection
        conn, addr = server_socket.accept()
        print(f"Connection established with {addr}")

        # Chat loop
        while True:
```

```
            # Receive a message from the client
            client_message = conn.recv(1024).decode()
            if client_message.lower() == "exit":
                print("Client has exited the chat.")
                break
            print(f"Client: {client_message}")

            # Send a message to the client
            server_message = input("You: ")
            conn.sendall(server_message.encode())
            if server_message.lower() == "exit":
                print("You exited the chat.")
                break

        # Close the connection
        conn.close()
        print("Connection closed.")
    except Exception as e:
        print(f"An error occurred: {e}")
    finally:
        server_socket.close()

# Start the server
if __name___== "__main__":
    chat_server()
```

CLIENT PROGRAM

```
import socket

def chat_client(host='localhost', port=12345):
    try:
        # Create a TCP socket
        client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # Connect to the server
        client_socket.connect((host, port))
        print(f"Connected to server at {host}:{port}")

        # Chat loop
        while True:
            # Send a message to the server
            client_message = input("You: ")
            client_socket.sendall(client_message.encode())
            if client_message.lower() == "exit":
                print("You exited the chat.")
                break

            # Receive a message from the server
            server_message = client_socket.recv(1024).decode()
```

```
            if server_message.lower() == "exit":
                print("Server has exited the chat.")
                break
            print(f"Server: {server_message}")

        # Close the connection
        client_socket.close()
    except Exception as e:
        print(f"An error occurred: {e}")

# Start the client
if __name___== "__main__":
    chat_client()
```

Sample Input & Output
Server Console
Server started at localhost:12345
Waiting for a client to connect...
Connection established with ('127.0.0.1', 54321)
Client: Hello, Server!
You: Hello, Client!
Client: How are you?
You: I'm fine. Type 'exit' to end the chat.

Client Console
Connected to server at localhost:12345
You: Hello, Server!
Server: Hello, Client!
You: How are you?
Server: I'm fine. Type 'exit' to end the chat.

Result

The Chat Client and Chat Server programs successfully exchange messages in real time using TCP sockets.

Ex.No.4 SIMULATION OF DNS USING UDP SOCKETS

AIM

To write a program in Python to simulate DNS (Domain Name System) resolution using UDP sockets.

ALGORITHM

SERVER-SIDE ALGORITHM

1. **Start the program**:
   - Import the socket module.
2. **Create a UDP socket**:
   - Use socket.socket() with AF_INET for IPv4 and SOCK_DGRAM for UDP.
3. **Bind the socket**:
   - Bind the socket to a specific host (e.g., localhost) and port (e.g., 5353, the DNS port).
4. **Create a DNS mapping**:
   - Maintain a dictionary mapping domain names to their corresponding IP addresses.
5. **Wait for client requests**:
   - Use recvfrom() to receive DNS queries from clients.
6. **Resolve the query**:
   - Look up the requested domain in the dictionary. If found, return the corresponding IP address; otherwise, return an error message.
7. **Send the response**:
   - Use sendto() to send the response back to the client.
8. **Repeat until stopped**:
   - Keep the server running to handle multiple queries.

Client-Side Algorithm

1. **Start the program**:
   - Import the socket module.
2. **Create a UDP socket**:
   - Use socket.socket() with AF_INET for IPv4 and SOCK_DGRAM for UDP.
3. **Send a query to the server**:
   - Use sendto() to send a domain name as a query to the DNS server.
4. **Wait for a response**:
   - Use recvfrom() to receive the response from the server.
5. **Display the response**:
   - Print the IP address or error message returned by the server.
6. **Close the socket**:
   - Terminate the connection after receiving the response.

Program

Server Program

```
import socket

def dns_server(host='localhost', port=5353):
    # Predefined DNS mapping (domain: IP address)
    dns_records = {
        "www.google.com": "142.250.74.14",
        "www.facebook.com": "31.13.71.36",
        "www.twitter.com": "104.244.42.129",
        "www.github.com": "140.82.114.4",
    }

    try:
        # Create a UDP socket
        server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

        # Bind the socket to the host and port
        server_socket.bind((host, port))
        print(f"DNS Server is running at {host}:{port}")

        while True:
            # Receive query from the client
            query, client_address = server_socket.recvfrom(1024)
            domain_name = query.decode()
            print(f"Received query for: {domain_name}")

            # Resolve the query
            ip_address = dns_records.get(domain_name, "Domain not found")

            # Send the response to the client
            server_socket.sendto(ip_address.encode(), client_address)
            print(f"Response sent: {ip_address}")
    except Exception as e:
        print(f"An error occurred: {e}")
    finally:
        server_socket.close()

# Start the DNS server
if __name__ == "__main__":
    dns_server()
```

Client Program

```
import socket

def dns_client(server_host='localhost', server_port=5353):
    try:
        # Create a UDP socket
        client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

        while True:
            # Input domain name from user
            domain_name = input("Enter domain name to resolve (or 'exit' to quit): ")
            if domain_name.lower() == "exit":
                print("Exiting DNS client.")
                break

            # Send query to the DNS server
            client_socket.sendto(domain_name.encode(), (server_host, server_port))

            # Receive the response from the server
            response, _ = client_socket.recvfrom(1024)
            print(f"IP Address: {response.decode()}")
    except Exception as e:
        print(f"An error occurred: {e}")
    finally:
        client_socket.close()

# Start the DNS client
if __name___== "__main__":
    dns_client()
```

Sample Input & Output
Server Console

DNS Server is running at localhost:5353
Received query for: www.google.com
Response sent: 142.250.74.14
Received query for: www.unknown.com
Response sent: Domain not found
Client Console

Result

The DNS simulation using UDP sockets successfully resolved domain names to IP addresses or returned appropriate error messages.

Ex.No.5 USE A TOOL LIKE WIRESHARK TO CAPTURE PACKETS AND EXAMINE THE PACKETS

Aim

To capture, filter, and analyze network packets using the Wireshark application.

Requirements

1. **Software**: Wireshark (latest version)
2. **Hardware**: Ethernet-enabled computer with internet access

Theory

Wireshark is a widely-used network protocol analyzer that lets you capture and interactively browse the traffic running on a computer network. It is used for network troubleshooting, analysis, and communication protocol development.

Steps to Perform the Experiment

---

1. Install Wireshark

- Download Wireshark from https://www.wireshark.org.
- Install it on your system with default options.

---

2. Launch Wireshark

- Open the **Wireshark** application.
- You will see a list of available network interfaces on your computer.

---

3. Start Packet Capture

1. Select the **Ethernet Interface**:
    - Choose the network interface corresponding to your Ethernet connection.
    - It may appear as Ethernet0, eth0, or similar depending on your system.
2. Start Capturing:
    - Click on the **Start** button (green shark fin icon) to begin packet capture.

---

4. Perform the Network Activity

- While capturing, perform a network activity, such as:
  - Running your **DNS program** or socket programs.
  - Browsing a website or downloading a file.

---

5. Stop Packet Capture

- After completing the activity, click on the **Stop** button (red square icon) to stop capturing packets.

Steps to Analyze the Captured Packets

1. Apply Filters

- Use the **Filter bar** at the top of the Wireshark window to filter specific types of packets.
- Common filters:
  - **DNS Traffic**: udp.port == 53
  - **HTTP Traffic**: http
  - **TCP Traffic**: tcp
  - **UDP Traffic**: udp
  - **Filter by IP Address**: ip.addr == <your-IP-address>

2. Examine the Packets

- Click on a packet in the **Packet List Pane** to view its details.
- The middle pane shows protocol details, and the bottom pane displays the raw packet data (hexadecimal and ASCII formats).

3. Look for Specific Protocol Details

- **DNS Analysis**:
  - Look for DNS queries and responses with domain names and IP addresses.
- **HTTP Analysis**:
  - Analyze GET and POST requests and their server responses.
- **TCP Analysis**:
  - Observe source and destination ports, flags, and handshake details.

Steps to Export Packet Data

- Go to **File > Save As** to save the captured packets for later analysis.

Sample Observations

DNS Traffic

- Query: Standard query A www.google.com
- Response: 142.250.74.14

HTTP Traffic

- Request: GET /index.html
- Response: HTTP/1.1 200 OK

TCP Traffic

- Source Port: 12345
- Destination Port: 80
- Flags: SYN, ACK

Result

The Wireshark tool was successfully used to capture, filter, and analyze network packets for DNS, HTTP, TCP, and UDP protocols.

Ex no 6 . Simulation of Address Resolution Protocol (ARP) Using Python and Cisco Devices

Aim

To simulate the Address Resolution Protocol (ARP) using Python, observe how it works to map an IP address to a MAC address in a local network, and capture the ARP packets using Cisco devices.

Requirements

1. **Software**:
   - Python 3.x
   - Cisco Packet Tracer (or real Cisco Router/Switch) for networking simulation.
   - Scapy Python library (for ARP simulation)
2. **Hardware**:
   - Ethernet-enabled computer with network access
   - Cisco Router/Switch (if using a physical network setup)
3. **Knowledge**:
   - Basic understanding of Cisco Networking concepts (routers, switches, IP addressing)

Theory

The **Address Resolution Protocol (ARP)** is a protocol used to resolve an IP address to its corresponding MAC address within the same local area network (LAN). Devices in the LAN use ARP to discover the MAC address associated with a known IP address.

- **ARP Request**: Sent by a device when it needs to find the MAC address corresponding to a given IP address.
- **ARP Reply**: Sent by the device that owns the IP address with its MAC address.

Python Program for ARP Simulation

1. Install Scapy

Before running the program, install the scapy library using the following command:

bash
Copy code
pip install scapy

2. ARP Simulation Program

python
Copy code

```
from scapy.all import ARP, Ether, srp

# Function to perform ARP request and capture response
def perform_arp_request(target_ip):
    # Create an ARP request to get the MAC address of the target IP
    arp_request = ARP(pdst=target_ip)

    # Create an Ethernet frame to wrap the ARP request
    broadcast = Ether(dst="ff:ff:ff:ff:ff:ff")

    # Combine the Ethernet frame with the ARP request
    arp_request_broadcast = broadcast/arp_request

    # Send the request and capture the response (timeout is set to 1 second)
    answered_list, unanswered_list = srp(arp_request_broadcast, timeout=1, verbose=False)

    return answered_list

# Function to display the results
def display_result(answered_list):
    print("IP Address\t\tMAC Address")
    print("-----------------------------------------------")

    for response in answered_list:
        ip = response[1].psrc  # IP address
        mac = response[1].hwsrc  # MAC address
        print(f"{ip}\t\t{mac}")

# Main function to take input and perform ARP request
if __name___== "__main__":
    target_ip = input("Enter the target IP address (e.g., 192.168.1.1): ")
    answered_list = perform_arp_request(target_ip)

    if answered_list:
        display_result(answered_list)
    else:
        print("No response received. Please check the target IP address or your network
connection.")
```

Steps to Perform the Experiment

1. Install Scapy

- Open a terminal or command prompt and install the scapy library using the command:

  bash
  Copy code

pip install scapy

2. Run the Python Program

- Save the above Python code as arp_simulation.py.
- Run the program using the following command:

  bash
  Copy code
  python arp_simulation.py

- The program will ask for an IP address to query (e.g., 192.168.1.1). Enter the IP address of a device within your local network.

3. Cisco Packet Tracer (If Using Cisco Simulation)

- **Set up Cisco Packet Tracer**: Create a simple network with a Cisco router or switch and connect some end devices (PCs or servers). Assign IP addresses to these devices within the same subnet.
- **Ping a Device in Cisco Packet Tracer**: On a Cisco PC, open the command prompt and ping another device (e.g., 192.168.1.1).
- **Observe ARP Behavior in Cisco**: After pinging, use the show ip arp command on a Cisco router or switch to observe the ARP cache. This will display the mapping of IP addresses to MAC addresses in the local network.

Analyze the Results

1. Check ARP Table in Cisco Device

- In Cisco Packet Tracer, you can use the following command to view the ARP table on a router or switch:

  bash
  Copy code
  show ip arp

- This will show the IP-to-MAC address mapping for devices that have been pinged within the same subnet.

2. Check ARP Table in Python Output

- The Python program will display the IP and corresponding MAC address for the given target IP after sending an ARP request. It will show something like:

  text
  Copy code
  IP Address MAC Address

192.168.1.1 00:1a:2b:3c:4d:5e

Sample Output

text
Copy code
Enter the target IP address (e.g., 192.168.1.1): 192.168.1.1
IP Address MAC Address

192.168.1.1 00:1a:2b:3c:4d:5e

Steps to Capture ARP Packets Using Wireshark

1. Set Up Wireshark

- Open **Wireshark** and start capturing packets on the correct network interface (Ethernet or Wi-Fi).
- Apply a filter to only show ARP packets by typing the following in the filter bar:

  text
  Copy code
  arp

- You will be able to observe ARP requests and replies being exchanged between devices.

2. Analyze ARP Packets

- **ARP Request**: Sent from a source IP to request the MAC address of the target IP.
- **ARP Reply**: Sent from the device with the target IP, containing its MAC address.

Result

The Address Resolution Protocol (ARP) was successfully simulated in Python, and ARP packets were analyzed using Wireshark and the Cisco Router/Switch. The IP address and corresponding MAC address were obtained.

6.B SIMULATION OF REVERSE ADDRESS RESOLUTION PROTOCOL (RARP) USING PYTHON

Aim

To simulate the Reverse Address Resolution Protocol (RARP) using Python and understand how a device can obtain its IP address based on its MAC address.

Requirements

1. **Software**:
   - Python 3.x
   - Scapy Python library (for RARP simulation)
2. **Hardware**:
   - Ethernet-enabled computer with network access
3. **Knowledge**:
   - Basic understanding of networking protocols (especially ARP and RARP)

Theory

The **Reverse Address Resolution Protocol (RARP)** is the reverse process of ARP. While ARP is used by devices to map an IP address to a MAC address, RARP allows a device to map its MAC address to an IP address. RARP is generally used by diskless workstations to find their IP address when they boot up, as they don't have the ability to store an IP address locally.

- **RARP Request**: A device sends a RARP request to a RARP server (typically a router or dedicated server) to obtain its IP address.
- **RARP Reply**: The server responds with the corresponding IP address for the given MAC address.

Python Program for RARP Simulation

1. Install Scapy

Before running the program, install the scapy library using the following command:

bash
Copy code
pip install scapy

2. RARP Simulation Program

python

```
from scapy.all import ARP, Ether, srp

# Function to perform RARP request and capture response
def perform_rarp_request(mac_address):
    # Create a RARP request to get the IP address for the given MAC address
    rarp_request = ARP(op=3, hwsrc=mac_address)

    # Create an Ethernet frame to wrap the RARP request
    broadcast = Ether(dst="ff:ff:ff:ff:ff:ff")

    # Combine the Ethernet frame with the RARP request
    rarp_request_broadcast = broadcast/rarp_request

    # Send the request and capture the response (timeout is set to 1 second)
    answered_list, unanswered_list = srp(rarp_request_broadcast, timeout=1, verbose=False)

    return answered_list

# Function to display the result of RARP request
def display_result(answered_list):
    print("MAC Address\t\tIP Address")
    print("-----------------------------------------------")

    for response in answered_list:
        mac = response[1].hwsrc  # MAC address
        ip = response[1].psrc  # IP address
        print(f"{mac}\t\t{ip}")

# Main function to take input and perform RARP request
if __name___== "__main__":
    mac_address = input("Enter the MAC address (e.g., 00:1a:2b:3c:4d:5e): ")
    answered_list = perform_rarp_request(mac_address)

    if answered_list:
        display_result(answered_list)
    else:
        print("No response received. Please check the MAC address or your network
connection.")
```

Steps to Perform the Experiment

1. Install Scapy

- Open a terminal or command prompt and install the scapy library using the command:

  bash
  Copy code
  pip install scapy

2. Run the Python Program

- Save the above Python code as rarp_simulation.py.
- Run the program using the following command:

  bash
  Copy code
  python rarp_simulation.py

- The program will ask for a MAC address to query (e.g., 00:1a:2b:3c:4d:5e). Enter the MAC address of a device in your local network.

3. Observe the Output

- The program will attempt to send a RARP request for the given MAC address and print the corresponding IP address if available.

Analyze the Results

- The **RARP Request** is sent to the network using the device's MAC address.
- The **RARP Reply** from the server (or router) provides the IP address mapped to that MAC address.
- If the MAC address exists in the RARP server's database, the corresponding IP address is displayed.

Sample Output
text
Copy code
Enter the MAC address (e.g., 00:1a:2b:3c:4d:5e): 00:1a:2b:3c:4d:5e
MAC Address IP Address
- -
00:1a:2b:3c:4d:5e 192.168.1.5

Steps to Capture RARP Packets Using Wireshark

1. Set Up Wireshark

- **Open Wireshark** and start capturing packets on the correct network interface (Ethernet or Wi-Fi).
- Apply a filter to only show ARP and RARP packets by typing the following in the filter bar:

  text
  Copy code
  arp or rarp

- You will be able to observe RARP requests and replies being exchanged between devices.

2. Analyze RARP Packets

- **RARP Request**: Sent from a device with a specific MAC address to request its IP address.
- **RARP Reply**: Sent from the **RARP** server, containing the IP address corresponding to the device's MAC address.

Result

The Reverse Address Resolution Protocol (RARP) was successfully simulated in Python, and RARP packets were analyzed using Wireshark. The MAC address was mapped to the corresponding IP address, as observed in the output of the Python program

Ex No. 7b SIMULATION OF CONGESTION CONTROL ALGORITHMS USING CISCO

Aim

To simulate and observe congestion control algorithms (such as TCP Tahoe, TCP Reno, and TCP Vegas) using Cisco Packet Tracer and analyze how these algorithms manage congestion in a network.

Requirements

1. **Software:**
   - Cisco Packet Tracer (for network simulation)
   - PC with Cisco Packet Tracer installed
2. **Hardware:**
   - Ethernet-enabled computers or routers
3. **Knowledge:**
   - Basic understanding of networking protocols, especially TCP congestion control algorithms.

Theory

Congestion control algorithms are used in networks to prevent packet loss and ensure efficient data transmission under varying network conditions. These algorithms adjust the data transmission rate based on feedback from the network to avoid congestion.

- **TCP Tahoe**: A basic congestion control mechanism that uses slow-start, congestion avoidance, and fast retransmit.
- **TCP Reno**: An enhanced version of TCP Tahoe that includes fast recovery to handle packet loss.
- **TCP Vegas**: An improved version of TCP Reno that uses the round-trip time (RTT) for better congestion control and avoids overloading the network.

In a typical network, packet loss, delay, and network congestion can significantly impact performance. TCP congestion control mechanisms help in dynamically adjusting the sender's transmission rate to alleviate congestion, minimize delays, and optimize throughput.

Steps to Simulate Congestion Control Algorithms in Cisco Packet Tracer

1. Set up the Network Topology

- Open **Cisco Packet Tracer** and create a network topology that includes:
  - At least two **PCs** connected via a **router**.

- Add some **switches** if necessary for larger networks.
- Ensure both PCs are on the same network (subnet).
- Configure basic IP addressing for all devices in the network.

2. Configure Devices

- **PC Configuration**:
  1. On each PC, configure the **IP address** and **subnet mask**.
  2. Ensure that the PCs can **ping** each other to confirm connectivity.
- **Router Configuration** (if needed for routing):
  1. Set up routing between subnets using static routes or a dynamic routing protocol like **RIP**.
  2. Configure the interfaces and IP addresses on the router.

3. Set up Traffic Flow

- To simulate congestion, generate traffic between the two PCs. This can be done using **ping** or by initiating **TCP connections** (e.g., web traffic or file transfers).
- For TCP traffic:
  - Open the **Command Prompt** on one of the PCs.
  - Use a command like ping -t 192.168.1.2 (replace with the target PC's IP address) to generate continuous traffic.
  - Alternatively, use a **web server** or **FTP server** on one of the PCs and use the **web browser** on the other to generate TCP traffic.

4. Implement and Observe Congestion Control Algorithms

- Cisco routers and switches automatically implement TCP congestion control algorithms. However, you can observe the behavior of these algorithms by:
  1. **Sending continuous traffic** between the two PCs.
  2. **Simulating congestion** by adding network delays or artificial bottlenecks (e.g., by limiting bandwidth or adding large amounts of traffic).
  3. **Observe Packet Drops** and Congestion Windows by using **debugging tools** in Cisco Packet Tracer.
- **To observe congestion control in action**, monitor the following:
  - **Slow Start**: In the initial phase, the sender gradually increases the transmission window.
  - **Congestion Avoidance**: Once the congestion window reaches a threshold, the sender adjusts the rate to avoid congestion.
  - **Packet Loss**: As the network experiences congestion, packets are dropped, and the sender reduces its window size to alleviate congestion.

5. Analyze Traffic

- Once the network is running and traffic is flowing, observe the **congestion window (CWND)** and **Round-Trip Time (RTT)**. You can monitor these parameters using the **Simulation Mode** in Cisco Packet Tracer.
- Use the **Command Prompt** to view changes in RTT as congestion increases.
- If packet loss occurs, observe the TCP behavior (slow start and congestion avoidance) and how the sender reduces the transmission rate.

Expected Observations

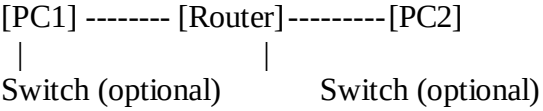
- **TCP Tahoe**: The window size increases exponentially at first (slow start) and then linearly (congestion avoidance). Upon packet loss, TCP Tahoe resets the window size to the initial value (slow start).
- **TCP Reno**: TCP Reno performs similarly to TCP Tahoe but with an additional **fast recovery** phase. After detecting a packet loss, Reno recovers faster by halving the congestion window size but not resetting it entirely.
- **TCP Vegas**: Vegas uses the round-trip time (RTT) to detect potential congestion early and adjusts the sending rate before packet loss occurs. It aims to maintain a low RTT by proactively reducing the window size.

Capturing and Monitoring Data Using Simulation Mode

1. **Switch to Simulation Mode** in Cisco Packet Tracer.
2. Start the **ping test** or **TCP communication** between PCs.
3. Observe the **packets** as they travel through the network.
4. Look for the **TCP flags** (e.g., SYN, ACK) and **sequence numbers**.
5. Use the **Event List** to track each packet and see the impact of congestion control on the flow.

Sample Topology

text
Copy code



Result

The simulation successfully demonstrated the congestion control algorithms (TCP Tahoe, TCP Reno, and TCP Vegas) in action. Observations of the TCP window size, RTT, and packet loss showed how these algorithms adapt to network congestion to ensure efficient data transfer.

Ex.No:8 STUDY OF TCP/UDP PERFORMANCE USING SIMULATION TOOL :

Aim

To study and compare the performance of TCP and UDP protocols in a network using Cisco Packet Tracer simulation.

Requirements

1. **Software**:
   - Cisco Packet Tracer (Network Simulator)
2. **Hardware**:
   - PCs, switches, and routers (simulated in Cisco Packet Tracer)
3. **Knowledge**:
   - Basics of TCP and UDP protocols
   - Configuration of IP addresses and devices in a network.

Theory

TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are two primary transport layer protocols.

1. **TCP (Transmission Control Protocol)**:
   - A connection-oriented protocol that ensures reliable data delivery.
   - Uses mechanisms like error checking, retransmission, and acknowledgment (ACK).
   - Best suited for applications requiring high reliability (e.g., file transfer, emails).
2. **UDP (User Datagram Protocol)**:
   - A connectionless protocol offering faster data transfer but without reliability guarantees.
   - Suitable for time-sensitive applications like video streaming or gaming.

Comparison:
TCP ensures reliable data transfer with higher latency, while UDP offers faster but less reliable data transfer. This simulation demonstrates these differences.

Steps to Simulate TCP and UDP Performance

1. Setup the Network Topology

- Open **Cisco Packet Tracer**.

- Create the following topology:

text
Copy code
[PC1] --- [Switch] --- [Router] --- [Switch] --- [PC2]

2. Configure the Network

- **IP Address Configuration**:
  - Assign IP addresses to all devices (PCs, routers) in the same subnet.
- Example configuration:
  - PC1: 192.168.1.2 /24
  - Router Interface 1: 192.168.1.1 /24
  - Router Interface 2: 192.168.2.1 /24
  - PC2: 192.168.2.2 /24
- **Static Routing** (if applicable):

Configure static routes on the router to enable communication between the subnets.

3. Generate Traffic Using TCP and UDP

- Use **Simulation Mode** in Cisco Packet Tracer to observe TCP and UDP traffic.
- **TCP Traffic**:
  - Configure a **web server** on PC2 and use a **web browser** on PC1 to access the server.
  - Alternatively, use a file transfer protocol (FTP) server.
  - Observe the 3-way handshake (SYN, SYN-ACK, ACK) and acknowledgment of packets.
- **UDP Traffic**:
  - Configure a **TFTP (Trivial File Transfer Protocol)** server on PC2 and use PC1 to transfer files.
  - Observe the lack of acknowledgment and retransmission.

4. Analyze Performance Parameters

- **Latency**:
  - Compare the delay experienced in TCP and UDP communication.
- **Packet Loss**:
  - Introduce network congestion or simulate packet loss to observe how TCP retransmits lost packets, while UDP does not.
- **Throughput**:
  - Measure the rate of successful data delivery in both protocols.
  - Use tools like **ping** or **continuous file transfers** to observe throughput.

5. Visualize Packets

- In **Simulation Mode**, observe:
  - TCP packets showing retransmission, acknowledgment, and congestion control.
  - UDP packets showing faster delivery but no reliability mechanisms.

Observations

Parameter	TCP	UDP
Reliability	Ensures reliable delivery	No reliability mechanism
Packet Delivery	Retransmits lost packets	Drops lost packets
Latency	Higher due to ACKs and retransmissions	Lower latency
Applications	File Transfer, Web Browsing	Video Streaming, Gaming

Result

The performance of TCP and UDP protocols was successfully studied and analyzed using Cisco Packet Tracer. The observations highlight the trade-offs between reliability and speed in TCP and UDP.

EX.NO.9A SIMULATION OF DISTANCE VECTOR ROUTING ALGORITHM

Aim

To simulate the Distance Vector Routing (DVR) algorithm using Python and observe the routing table updates and shortest path determination using Cisco Packet Tracer.

Requirements

1. **Software**:
    - Python (Version 3.x)
    - Cisco Packet Tracer
2. **Hardware** (Simulated in Cisco Packet Tracer):
    - PCs, switches, and routers
3. **Knowledge**:
    - Basics of Distance Vector Routing Algorithm
    - Configuration of routing tables in Cisco devices.

Theory

Distance Vector Routing Algorithm:

- A routing protocol where each router shares its routing table with directly connected neighbors periodically.
- It uses the **Bellman-Ford algorithm** to compute the shortest path.
- Key steps include:
    1. Initialization: Each node maintains a routing table.
    2. Table Exchange: Nodes exchange routing information with neighbors.
    3. Update: Nodes update their tables based on received information.
    4. Convergence: Routing tables eventually stabilize.

Python Simulation

Algorithm:

1. Initialize the network as a graph with weighted edges representing the cost of links.
2. Assign each router a routing table initialized with direct neighbor costs.
3. Implement the Distance Vector Algorithm to update routing tables iteratively.
4. Print the final routing tables once convergence is achieved.

Python Code

python
Copy code

```
# Distance Vector Routing Algorithm Simulation in Python
def distance_vector_routing(graph, num_nodes):
    # Initialize routing tables
    routing_table = {node: {neighbor: float('inf') for neighbor in range(num_nodes)} for node
in range(num_nodes)}
    for node in graph:
        for neighbor, cost in graph[node]:
            routing_table[node][neighbor] = cost
        routing_table[node][node] = 0

    # Distance Vector Algorithm
    updated = True
    while      updated:
        updated = False
        for node in range(num_nodes):
            for neighbor, cost in graph[node]:
                for target in range(num_nodes):
                    if routing_table[neighbor][target] + cost < routing_table[node][target]:
                        routing_table[node][target] = routing_table[neighbor][target] + cost
                        updated = True

    # Display routing tables
    for node in range(num_nodes):
        print(f"Routing Table for Router {node}:")
        for target, cost in routing_table[node].items():
            print(f"To {target} via {node} = {cost}")
        print()

# Example graph (Node: [(Neighbor, Cost)])
graph = {
    0: [(1, 1), (2, 4)],
    1: [(0, 1), (2, 2), (3, 6)],
    2: [(0, 4), (1, 2), (3, 3)],
    3: [(1, 6), (2, 3)]
}
num_nodes = 4

# Run the simulation
distance_vector_routing(graph, num_nodes)
```

Cisco Packet Tracer Implementation

Steps to Simulate DVR in Cisco Packet Tracer

1. **Setup the Topology**:

- Create a network with 4 routers connected in a mesh topology as shown below:

  lua
  Copy code
  R1 --- R2
  | |
  R3 --- R4

2. **Assign IP Addresses**:
   - Configure IP addresses on the interfaces of each router.
3. **Enable RIP (Routing Information Protocol)**:
   - On each router, configure RIP as follows:

   bash
   Copy code
   Router> enable
   Router# configure terminal
   Router(config)# router rip
   Router(config-router)# version 2
   Router(config-router)# network <subnet-ID>
   Router(config-router)# exit
   Router(config)# exit

   - Replace <subnet-ID> with the appropriate subnets for each router.
4. **Observe Routing Table Updates**:
   - Use the show ip route command on each router to view the routing table updates.
   - Verify that the routers share their routing tables and determine the shortest path.

Observations

Scenario	Routing Table Updates	Shortest Path Determined
Initial Network Setup	Proper sharing of tables	Optimal path is chosen
Link Failure Simulated	RIP recalculates paths	Convergence to a new path

---

Result

The Distance Vector Routing Algorithm was successfully simulated using Python and Cisco Packet Tracer. Routing tables were updated dynamically, and shortest paths were determined under varying network conditions.

9.B Simulation of Link State Routing Algorithm Using Python

Aim

To simulate the Link State Routing (LSR) algorithm using Python and observe how the shortest paths are calculated and routing tables are updated dynamically.

Requirements

1. **Software**:
   - Python (Version 3.x)
2. **Knowledge**:
   - Basics of Link State Routing Algorithm
   - Understanding of Dijkstra's Algorithm for shortest path computation

---

Theory

Link State Routing Algorithm:

- A routing protocol where each router knows the entire topology of the network.
- Uses **Dijkstra's Algorithm** to compute the shortest path from the source to all other nodes.
- Steps:
  1. Each router discovers its neighbors and measures the cost to reach them.
  2. The router constructs a **Link State Packet (LSP)** containing the costs of its directly connected links.
  3. LSPs are exchanged with all other routers in the network.
  4. Each router uses the collected LSPs to construct a complete topology map.
  5. The shortest path to all nodes is calculated using Dijkstra’s algorithm.

Python Simulation

Algorithm:

1. Represent the network as a graph using an adjacency matrix or dictionary.
2. Use Dijkstra's algorithm to find the shortest paths from the source node to all other nodes.
3. Display the shortest paths and routing tables for all nodes.

Python Code

Sample Output

For the above graph, the program will display routing tables with the shortest paths and costs for each node.

Procedure

1. **Understand the Graph Representation**:
   - Create a graph where nodes represent routers, and edges represent the cost between connected routers.
2. **Run the Python Code**:
   - Execute the Python script to simulate the Link State Routing algorithm.
3. **Observe the Output**:
   - Verify the correctness of the shortest paths and costs displayed in the routing tables.

Observations

Source Node	Destination Node	Shortest Path	Cost
0	1	0 → 1	2
0	2	0 → 1 → 2	3
0	3	0 → 1 → 2 → 3	6
...	...	...	...

Result

The Link State Routing Algorithm was successfully simulated using Python, and the shortest paths and routing tables were displayed for each node.

Ex No 10 .Simulation of Error Correction Code (Like CRC)

Aim

To simulate the Cyclic Redundancy Check (CRC) error correction code using Python and verify its ability to detect errors in transmitted data.

Requirements

1. **Software**:
   - Python (Version 3.x)
2. **Knowledge**:
   - Basics of Error Detection and Correction
   - Understanding of Cyclic Redundancy Check (CRC) Algorithm

---

Theory

Error Detection and Correction:
Error correction codes are used to detect and correct errors in transmitted data to ensure data integrity.

Cyclic Redundancy Check (CRC):

- CRC is a method used to detect errors in data transmission.
- It works by dividing the input message by a generator polynomial and appending the remainder (CRC checksum) to the message.
- At the receiver's end, the same polynomial is used to check for errors.

Steps:

1. Append a number of zeros equal to the degree of the polynomial to the input message.
2. Divide the message by the generator polynomial using XOR operations.
3. The remainder is the CRC checksum.
4. Transmit the original message along with the checksum.
5. The receiver performs the division again. If the remainder is 0, the message is error-free.

Python Simulation

Algorithm

1. Input the data message and the generator polynomial.
2. Append zeros to the message (based on the polynomial degree).
3. Perform division using XOR to compute the CRC checksum.
4. Simulate transmission of the message and append the checksum.
5. At the receiver, verify the message by recalculating the remainder.

Python Code

python

Copy code

```
def xor_operation(a, b):
    """Perform XOR operation between two binary strings."""
    result = []
    for i in range(1, len(b)):
        if a[i] == b[i]:
            result.append('0')
        else:
            result.append('1')
    return ''.join(result)

def divide_data(data, generator):
    """Perform division of data by generator polynomial."""
    pick = len(generator)
    temp = data[:pick]

    while pick < len(data):
        if temp[0] == '1':
            temp = xor_operation(generator, temp) + data[pick]
        else:
            temp = xor_operation('0' * pick, temp) + data[pick]
        pick += 1

    if temp[0] == '1':
        temp = xor_operation(generator, temp)
    else:
        temp = xor_operation('0' * pick, temp)

    return temp

def crc(data, generator):
    """Generate CRC checksum."""
    # Append zeros to the data
    l_gen = len(generator)
    data = data + '0' * (l_gen - 1)
    remainder = divide_data(data, generator)
    return remainder
```

```
def main():
    # Input data and generator polynomial
    data = input("Enter the data (binary): ")
    generator = input("Enter the generator polynomial (binary): ")

    # Compute CRC
    checksum = crc(data, generator)
    print(f"CRC Checksum: {checksum}")

    # Simulate transmission
    transmitted_data = data + checksum
    print(f"Transmitted Data: {transmitted_data}")

    # Receiver validation
    remainder = divide_data(transmitted_data, generator)
    print(f"Remainder at Receiver: {remainder}")

    if int(remainder) == 0:
        print("No error detected in the transmitted data.")
    else:
        print("Error detected in the transmitted data.")

if __name___== "__main__":
    main()
```

Sample Input and Output

Input:

- Data: 1101011011
- Generator Polynomial: 10011

Output:

- CRC Checksum: 1100
- Transmitted Data: 11010110111100
- Remainder at Receiver: 0000
- Result: No error detected in the transmitted data.

Procedure

1. **Understand CRC Algorithm**:
    - Understand how the polynomial division is performed in binary.
2. **Run the Python Code**:
    - Enter the binary data and generator polynomial as input.
3. **Simulate Transmission**:

- Observe the CRC checksum and transmitted data.

4. **Verify at Receiver**:
   - Check if the remainder at the receiver end is zero.

Observations

Data	Generator Polynomial	CRC Checksum	Transmitted Data	Remainder	Result
1101011011	10011	1100	11010110111100	0000	No error detected
1101011011	10011	1100	11010110111101	0001	Error detected in data

---

Result

The simulation of the Cyclic Redundancy Check (CRC) algorithm was successfully implemented using Python, and it was able to detect errors in the transmitted data.